

Gymnázium J. K. Tyla



Agentické inženýrství ve vývoji softwaru

Návrh a implementace DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnáziu J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

.....

Podpis autora práce:

Anotace

Práce představuje agentickou umělou inteligenci jako poslední krok přechodu od konverzační asistence k delegovanému vývoji a vymezuje pojem agentického inženýrství. Ukazuje, že jádrem této disciplíny není schopnost jazykového modelu, ale návrh systému, v němž model pracuje, a že pojem software factory, zavedený již v roce 1968, zažívá právě prostřednictvím agentů znovu. Výzkumná otázka zní, které principy musí agentický systém splnit, aby vykonával inženýrskou práci. Hypotéza, kterou práce v praktické části ověřuje, na ni odpovídá: praktická autonomie je vlastností návrhu systému, který práci řídí, a nikoli vlastností modelu, který v něm pracuje. Praktická část popisuje DarkFactory, záměrně minimální produkční softwareovou továrnu, v níž je GitHub vývojovým prostředím i trvalým stavem a pythonovský řadič spolu s produkčními harnessy realizuje interpretaci, plánování, implementaci, revizi a integraci změn. Strukturu, která práci řídí, navrhuje autor sám. Úsudek o tom, co je v jednotlivém kroku správné, však přebírá od smyčky, kterou nevlastní — a právě v tomto rozdílu leží hranice, za kterou teprve vlastní harness pomůže.

Klíčová slova: coding agents; software factory; AI; Agentic Engineering; harness

Abstract

This thesis presents agentic artificial intelligence as the latest step in the transition from conversational assistance to delegated development, and establishes the term agentic engineering. It argues that the core of the discipline is not the capability of a language model but the design of the system in which the model operates, and that software factory, a concept introduced in 1968, is being revived precisely through agents. The research question asks which principles an agentic system must satisfy in order to perform engineering work, and the hypothesis tested in the practical part answers it: practical autonomy is a property of the design of the system that governs the work, not of the model operating inside it. The practical part describes DarkFactory, a deliberately minimal production software factory in which GitHub is the development environment and the durable state, GitHub Actions the execution environment, a container the isolation layer, and a Python runner working with production harnesses performs intake, planning, implementation, review and integration. The structure that governs the work is the author's own; the judgement of what is correct in any single step is taken over from a loop he does not own — and beyond that difference lies the boundary that only a harness of one's own helps to cross.

Keywords: coding agents; software factory; AI; Agentic Engineering; harness

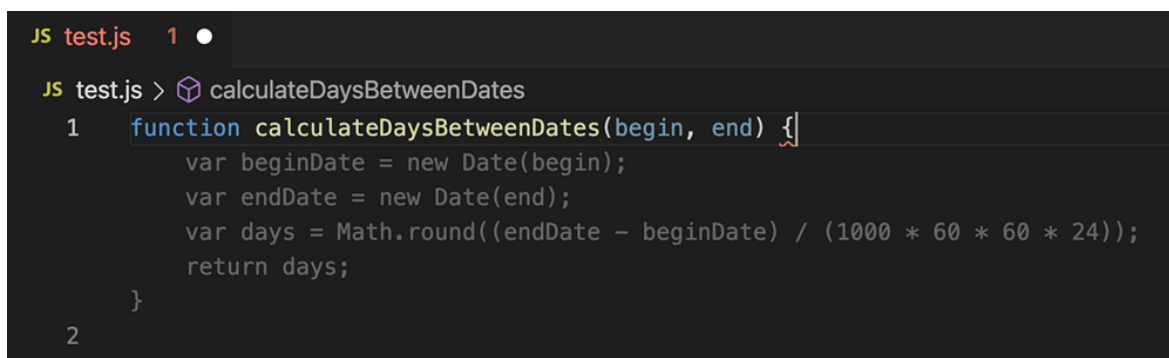
Obsah

Prohlášení	2
1 Úvod	5
1.1 Motivace: Vývoj a adopce generativní AI	5
1.2 Cíl, výzkumná otázka, hypotéza a vymezení	8
1.3 Terminologie	9
2 Teoretická část	10
2.1 Agent: Co to je a jak funguje	10
2.1.1 Context window a kompakce	11
2.1.2 Smyčka (Agent Loop)	11
2.1.3 Dovednosti a nástroje (Skills & Tools)	12
2.2 Agentické inženýrství	13
2.2.1 Zadání a plán (Prompt & Plan)	13
2.2.2 Orchestrace a lidská integrace (Orchestration & HITL)	13
2.3 Softwareová továrna	16
3 Praktická část	19
3.1 Metodika	19
3.1.1 Tři kritéria	20
3.1.2 Otevřená otázka: složení vrstev	21
3.2 Architektura produkčního běhu	22
3.3 Interpretace a plánování požadavku	24
3.4 Implementace a automatická revize	27
3.5 Zpětná vazba, schválení a úklid	29
4 Závěr	31
4.1 Zjištění	31
4.2 Diskuse	33
4.2.1 Omezení výzkumu	34
4.3 Shrnutí	35
Seznam zdrojů	37
Seznam součástí textu	43

1 Úvod

1.1 Motivace: Vývoj a adopce generativní AI

Nástroje založené na jazykových modelech se v krátké době drasticky zlepšily a jejich vývoj lze členit do tří stupňů. Nejprve doplňovaly kód v editoru (1), potom přišly konverzační chatboty (2), v nichž model sestavuje odpověď, ale nástroje mu zpravidla nebyly k dispozici, takže i nadále všechno provedl uživatel.



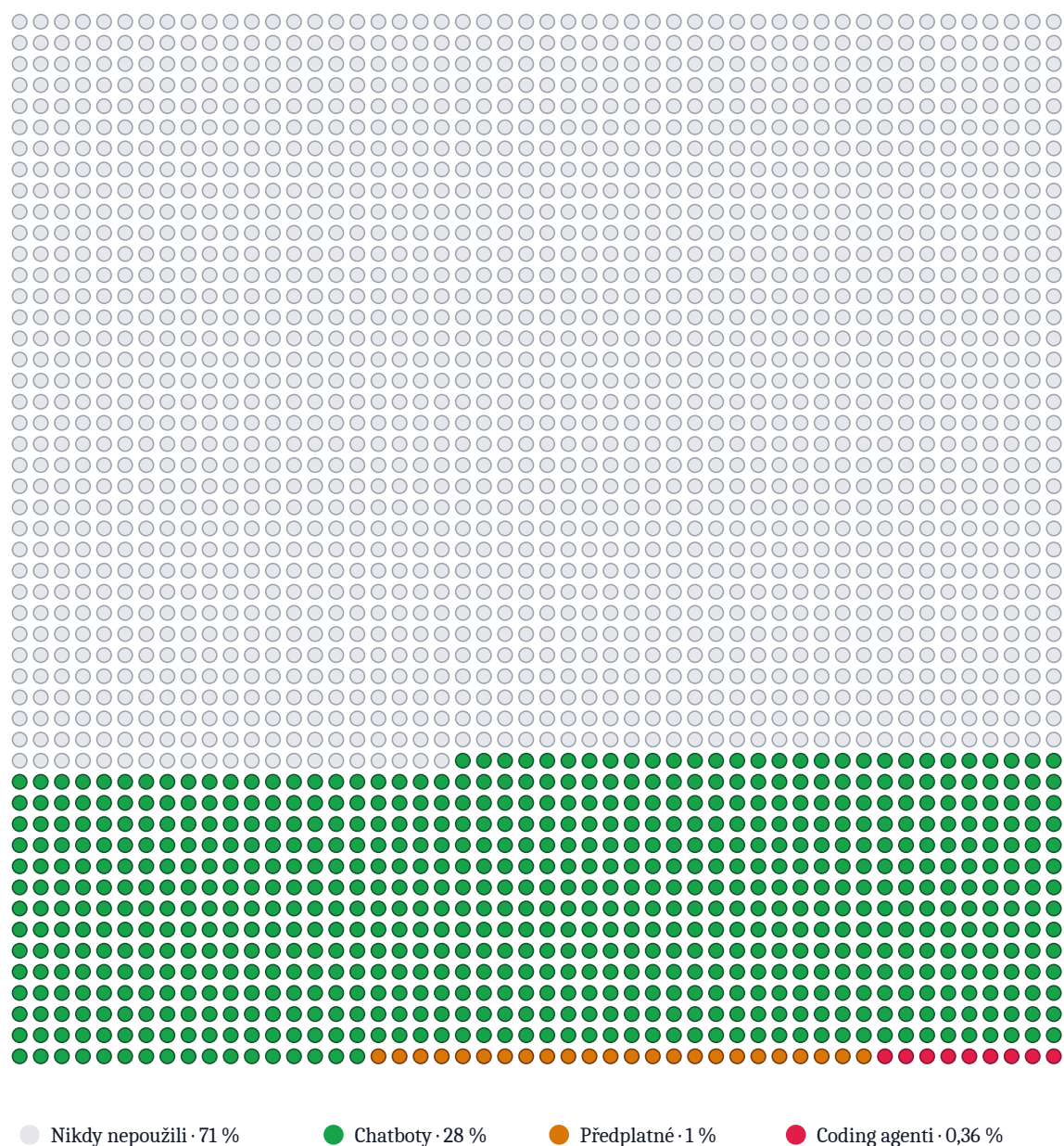
```
JS test.js 1 ●
JS test.js > calculateDaysBetweenDates
1 function calculateDaysBetweenDates(begin, end) {
    var beginDate = new Date(begin);
    var endDate = new Date(end);
    var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));
    return days;
2 }
```

Obrázek 1: Doplňování kódu přímo v editoru: model navrhuje pokračování řádku, které člověk přijme nebo odmítne (1).

Až třetí stupeň, *coding agenti* (3; 4; 5), dostal přístup k souborům, příkazům a běhovému prostředí — a tím se poprvé změnila věta, kdo pracuje.

Tuto hranici lze pojmenovat přesněji: **agent je model plus harness** (6). Harness je všechn kód, konfigurace a vykonávací logika, která není samotným modelem. Model sám o sobě neumí udržet stav mezi kroky, spustit kód, přistupovat k údajům, které se po jeho tréninku změnil, ani připravit si prostředí — a právě to všechno mu musí dodat harness. Hrubý model tedy agentem je teprve tehdy, když mu harness dodá stav, vykonávání nástrojů, zpětné vazby a vynutitelná omezení.

Praktickou hranicí mezi konverzačním chatbotem a agentem je právě míra delegovaného provádění. Samotná **inference** však nemění soubory, nespouští příkazy ani neuchovává stav mezi kroky. Tyto činnosti zajišťuje harness, který modelu zpřístupňuje nástroje, vyřizuje oprávnění a sestavuje prompt z přepisu konverzace. Rozdělení modelu a harnessu je tedy věcí odpovědnosti, nikoli pouhé implementace: první navrhuje, druhý jedná.



Obrázek 2: Odhad rozdělení uživatelů generativní AI podle typu (7).

Rozsah veřejného použití je přitom stále úzký. Jeden ze zveřejněných odhadů klade počet uživatelů chatbotů na 28 % populace a pravidelné užití *coding agentů* na 0,36 % světové populace (7); Obrázek 2 tyto dvě skupiny odlišuje.

Aby takový agent mohl na projektu pracovat, nestačí generovat odpovědi. Potřebuje kontext z repozitáře, přístup k prostředí, nástroje pro spouštění příkazů, stav, který přežije jednotlivé kroky, a vymezený bod, v němž člověk rozhodne o přijetí výsledku (8; 9). Všechno z toho dodává vrstva, která model pouze obaluje. Rozdíl mezi schopností modelu a návrhem systému, v němž model pracuje, je předmětem celé práce; význam pojmů, které ho vyjadřují a které se v češtině dosud neustálily, je sebrán v Kapitola 1.3.

Harness-Bench prošel 5 194 běhových trajektorií a zjistil, že výslednost se podle dvojice model-harness mění výrazně, takže schopnost agenta se má podávat na úrovni této dvojice a nikoli samotného modelu (10). HarnessX změřil průměrné zlepšení o 14,5 % na pěti benchmarkech, největší tam, kde byly výchozí výsledky nejhorší (11). Ding a jeho spolupracovníci ukázali, že část toho, co harness přidává k výsledku, lze po tréninku převést do parametrů modelu (12). Thangarajah a jeho spolupracovníci ukázali opačnou věc: co se převede, převádí se pouze v konvencích toho jediného scaffoldu, na němž se model trénoval (13). Představitelná práce, která architekturu rozkládá na vrstvy a mluví o ní jako o předmětu vlastního bádání, je poziční a dodává k tomu vlastní referenční implementaci, takže je citovatelná za slovník a za rámcování, nikoli za měření (14).

Tvrzení, že návrh systému je důležitější než schopnost modelu, samo o sobě neplatí; pole, které této práci předchází, samo píše, že další pokrok bude záviset *stejně* na návrhu systému jako na silnějších modelech, a nikoli místo nich (14). Stejně tak není doloženo, že se návrh systému od modelu oddělit nedá. Zajímavé není, co tvrzení říkají jednotlivě, ale v čem se liší: co lze do modelu přenést, co zůstává venku a podle čeho se o tom rozhoduje. To je předmětem teoretické části, která proto nepočítá s tím, že by měla vyhrát, ale s tím, že bude muset říct, kde se shoduje a kde ne.

1.2 Cíl, výzkumná otázka, hypotéza a vymezení

Rozhodující součástí agentického vývoje softwaru není schopnost jazykového modelu, ale návrh systému, v němž model pracuje. Cílem práce je tento názor vyargumentovat z dostupných zdrojů a prověřit na záměrně minimální implementaci, že právě tento návrh je nositelem schopnosti, o níž mluvíme.

Výzkumná otázka práce zní: *Které principy musí agentický systém splnit, aby vykonával inženýrskou práci?* Inženýrskou prací se zde rozumí změna repozitáře, kterou může jiný člověk než její autor přezkoumat a sloučit, aniž by musel agenta na cokoli ptát; v tomto smyslu je měřena schopnost systému a v tomto smyslu je brána člověka nezaměnitelnou. Sloveso *musí* je přitom míněno podmíněně: nutné jsou ty principy, které samotný model bez zásady okolí neposkytne. Které to jsou, se s rostoucí schopností modelů posouvá — a právě tato posunutost, nikoli její vymizení, je předmětem dalšího argumentu práce.

Práce předpokládá tuto hypotézu:

Praktická autonomie je vlastností návrhu systému, který práci řídí, a nikoli vlastností modelu, který v něm pracuje. Strukturu, kterou práce popisuje, autor navrhuje sám; úsudek o tom, co je v jednotlivém kroku správné, však přebírá od smyčky, kterou nevlastní — a právě v tomto rozdílu leží hranice, za kterou teprve vlastní harness pomůže.

Hypotéza se ověřuje v praktické části a z jejího výsledku práce vychází.

Praktická část popisuje DarkFactory, záměrně minimální produkční pipeline pro AI-asistovaný softwarový vývoj (15). Její běh, brány a uložený stav tvoří harness této práce; jako vykonávací engine používá produkční harnessy, protože to je pro popsanou míru detailu nejjednodušší. Z toho plyne úzká, ale zřetelně formulovaná otázka, kterou metodická část ponechává otevřenou: protože vnější vrstva je sama o sobě harnessem, setkává se model během jednoho běhu se dvěma soubory konvencí současně — s konvencemi této vrstvy a s konvencemi harnessu, který modelové kroky vykonává. Kterých konvencí model v takovém uspořádání následuje a co vnější vrstva stojí, je otázka, na kterou teoretická část neodpovídá.

Rozsah práce je záměrně úzký a odpovídá jedné revizi repozitáře: popisuje počáteční implementaci, v níž je agentní smyčka (*Agent Loop*) provedena cizím nástrojem. Její rozsah slouží jako základ, na němž navazuje práce následující.

1.3 Terminologie

Překlad anglických výrazů v této práci sleduje jedno pravidlo. Co anglicky nazývá **agent**, je česky **agentní** — agent, agentní krok, agentní smyčka. Co anglicky nazývá **agentic**, je česky **agentické** — agentický systém, agentické inženýrství. Podstatné jméno **agent** se tedy nepřekládá, protože označuje samotného jednajícího, zatímco přípona **-ic** znamená majícího schopnost jednat, nikoli jednajícího (16). Podle téhož pravidla je *Agent Loop* česky *agentní smyčka*, protože jde o smyčku agenta, a nikoli o vlastnost systému.

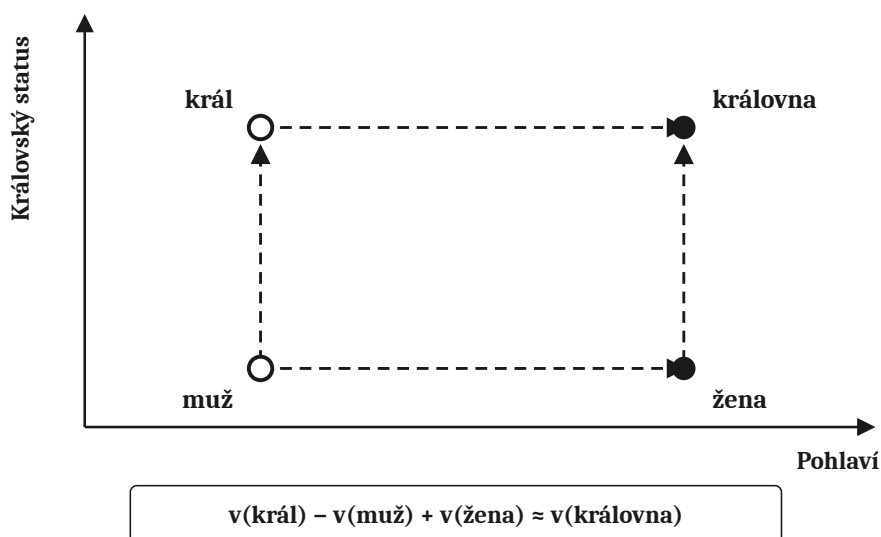
2 Teoretická část

2.1 Agent: Co to je a jak funguje

Současný jazykový model stojí na architektuře **Transformer**, kterou představil Google v roce 2017 v jejich nyní proslulé práci „*Attention Is All You Need*“ (17). Místo zpracovávání tokenů jeden po druhém přiřazuje architektura význam každému tokenu současně se všemi ostatními. Mechanismus, na kterém je architektura postavená, se nazývá **attention** a je dodnes používán i v pozdějších generacích modelů (18). Přitom ke každému tokenu připočítá vážený součet hodnot ostatních tokenů, takže jeho reprezentace nese informaci z celého kontextu a vzdálenost mezi pozicemi nemusí být pevně daná jejich pořadím. Právě to dovoluje zpracovat kontext najednou a vyhovět dnešním požadavkům na délku a složitost konverzace.

Na této architektuře je založen i jazykový model (**LLM**), který předpovídá další token na základě toho, co je před ním obsaženo v **kontextu**. Při **inferenci** model zpracuje obsah kontextového okna a vytvoří posloupnost výstupních tokenů.

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy v prostoru vektorů. Známým příkladem je vztah mezi vektory slov *král*, *královna*, *muž* a *žena* (19). Tento vztah schematicky znázorňuje Obrázek 3.



Obrázek 3: Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov *král*, *královna*, *muž* a *žena* (19).

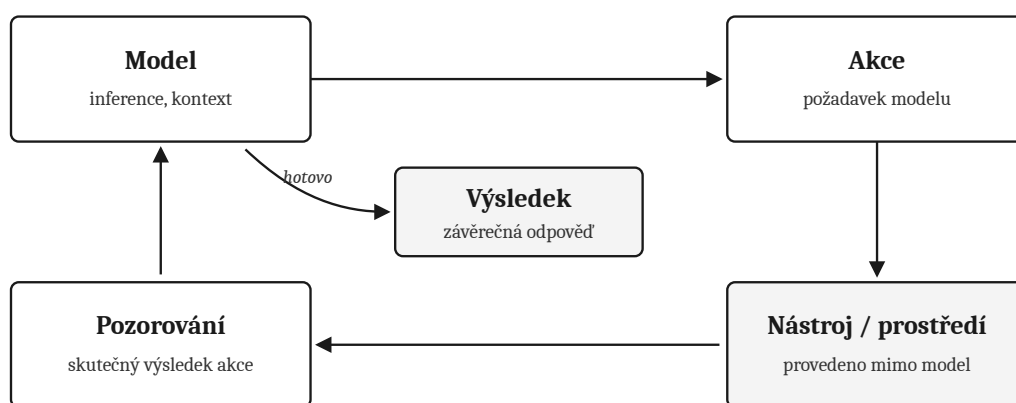
2.1.1 Context window a kompakce

Kontextové okno zahrnuje pracovní kontext jednoho volání modelu. Může obsahovat instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho kapacita však sama o sobě nezaručuje, že model všechny podstatné informace správně využije: úspěšnost jejich vybavení závisí také na umístění v kontextu a může s rostoucí délkou vstupu klesat (20). Toto postupné zhoršování práce s nahromaděným kontextem se označuje jako **context rot** (21).

Kompakce (*compaction*) po překročení stanoveného limitu nahrazuje starší průběh strukturovaným souhrnem klíčových rozhodnutí a dosažených výsledků. Do dalšího volání tak není nutné vkládat celý přepis předchozí interakce (21).

2.1.2 Smyčka (Agent Loop)

Základním mechanismem agentického systému je **agentní smyčka**, tedy konkrétní implementace vzoru **ReAct** (*Reasoning and Acting*) (22) v daném harnessu: model střídá uvažování s akcí a harness mezi jednotlivými kroky vrací pozorování, čímž z jednotlivé generace vzniká souvislá konverzace (23). V každém kroku model nejprve zdůvodní, co hodlá udělat, a vyjádří to jako požadavek na nástroj. Harness akci provede v běhovém prostředí a výsledek vrátí modelu jako pozorování; to se připojí k přepisu konverzace a cyklus se opakuje. Smyčka končí teprve tehdy, když model místo dalšího požadavku na nástroj vydá závěrečnou odpověď. Právě tím se smyčka liší od prosté posloupnosti promptů, v níž by model nemohl poznat, zda předchozí krok vůbec uspěl; průběh shrnuje Obrázek 4.



Harness přidává každý krok do přepisu konverzace a výsledek nástroje vrací modelu jako nové pozorování.

Obrázek 4: Agentní smyčka ReAct: model navrhne akci, harness ji provede v běhovém prostředí a pozorování se vrací do dalšího kroku. Ukončení nastává, když model místo další akce vydá závěrečnou odpověď (22).

2.1.3 Dovednosti a nástroje (Skills & Tools)

Zadání určuje, co má agent udělat; co mu je dovoleno udělat, určuje harness. Jde o dvě roviny,

které se dají měnit nezávisle, a odpovídají běžnému výčtu součástí harnessu (6):

- **Nástroje** umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy.
- **Skills** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy (24).
- **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit deterministickou kontrolu (25).
- **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient–server (26).

Instrukce lze uchovat ve standardizovaném souboru **AGENTS.md** (27) přímo v repozitáři (Anthropic ojedinele využívá **CLAUDE.md**). Skilly, skripty a hooky lze uchovat pod složkou **.agents/** v projektu nebo v konfigurační složce harnessu.

Tato dělba má přímý důsledek pro praxi. Oprávnění udělená nástroji a rozsah zadání se dají omezovat nezávisle: lze agentovi odebrat právo měnit cokoli mimo vyjmenované soubory, aniž by se změnil úkol, a lze zúžit úkol, aniž by mu přibyla schopnost. Kontrola, která by tyto dvě roviny zaměnila, by pak nedokázala říct, zda selhal úkol, nebo oprávnění.

2.2 Agentické inženýrství

Agentické inženýrství (**Agentic Engineering**) označuje soubor postupů, jimiž se vývoj softwaru pomocí coding agentů stává účinným, kontrolovaným, opakovatelným a škálovatelným (28). Jeho předmětem není samotný agent ani jeho model, ale systém, v němž agent pracuje: zadání, omezení, nástroje, pravidla integrace a odpovědnost člověka. Odtud také jeho popis jako archetypu **agentického inženýra**, jehož přidanou hodnotu už netvoří psaní kódu, ale formulace zadání, řízení agentních běhů a kritické posouzení strojem vytvořených výstupů (29). Mezi hlavní oblasti patří: **Prompt engineering** (30), **Context engineering** (21), **Harness engineering** (8; 31), **Loop Engineering** (32) a **Workflow/Graph Engineering** (33). Cílem je, aby vývojář mohl efektivně a kontrolovaně delegovat dílčí úkoly agentovi, aniž by ztratil přehled o záměru, rozsahu a kvalitě výsledku.

2.2.1 Zadání a plán (Prompt & Plan)

Spolehlivé delegování práce začíná explicitním vymezením cíle, rozsahu, omezení a podmínek přijetí. Specifikace popisuje nejen požadovaný výsledek, ale také části systému, které se měnit nemají, a způsob, jakým bude výsledek ověřen. Tento přístup, označovaný jako **spec-first** nebo *spec-driven development*, dává agentovi před implementací měřitelné hranice a člověku podklad pro posouzení výsledku. Přitom jde o postup používaný i v klasickém vývoji softwaru.

V praxi to znamená, že se plán nevzniká až během práce agenta, ale před ní: jeho znění se nejprve dohodnou s člověkem, zapíše do souboru a teprve potom se agent pustí do změn.

Prompt engineering se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku (30). **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici (21).

2.2.2 Orchestrace a lidská integrace (Orchestration & HITL)

Pokud mají podúlohy jasné hranice a jejich výsledky lze znovu integrovat, rozsáhlou úlohu lze rozdělit do více agentních běhů. Ve vzoru **coordinator/subagent** koordinátor deleguje dílčí úkol specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek. Nezávislé podúlohy mohou zpracovat paralelní pracovníci. Claude Code jde dál: u dynamických workflow Claude napíše skript, který runtime vykonává na pozadí. Smyčku, větvení i mezivýsledky pak drží skript místo kontextu modelu, takže plán je program, který lze přechít a znovu spustit (34).

V praxi se to projevuje v několika ustálených vzorech:

- **Workflow graph** předem určuje závislosti, pořadí a větvení fází (33).

- **Agent Swarm** dynamicky rozkládá úlohu na heterogenní podproblémy a spouští specializované agenty paralelně pod řízením orchestrátoru (35).
- **Goal loop** je nadřazená řídicí smyčka: po dílčím dokončení ReAct smyčky se pozorovaný stav porovná s cílem a podmínkami přijetí a běh skončí, nebo pokračuje další iterací. Vzor pochází z tzv. **Ralph loop**, který v roce 2025 popsal Geoffrey Huntley jako smyčku, jež opakovaně spouští agenta se stejným zadáním a stav si drží v pracovním stromu, nikoli v přepis konverzace (36). Dnes je to běžná funkce Claude Code i Codexu (32; 37).
- **human-in-the-loop (HITL)** označuje bod, v němž se do automatizované smyčky doplní kontrolní brány vyžadující explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak může ponechat agentovi ohraničenou implementační práci a současně si zachovat odpovědnost za záměr a integraci.

Vzorec koordinátor a subagent je běžně dostupný i v komerčních CLI harnessích, jak ukazuje Obrázek 6.

Phases	CodeReview · 16 agents
✓ Foundation 8/8	✓ cloud:security claude-codex-pro 47.2k tok · 15 tools · 1m 32s
✓ AcceptanceCriteria 4/4	✓ cloud:database claude-deepseek-pro 141.9k tok · 59 tools · 7m 14s
✓ CodeReview 16/16	✓ cloud:python Sonnet 4.6 120.4k tok · 39 tools · 12m 38s
✓ Verify 39/39	✓ cloud:stripe-deep claude-deepseek-pro 133k tok · 46 tools · 5m 5s
5 Completeness 3/4	✓ cloud:signup-deep claude-deepseek-pro 127.2k tok · 46 tools · 5m 58s
6 Synthesis	✓ gubbi:security claude-codex-pro 34.9k tok · 17 tools · 1m 14s
	✓ gubbi:database claude-deepseek-pro 117.9k tok · 73 tools · 7m 36s
	✓ gubbi:mcp-deep claude-deepseek-pro 113.1k tok · 48 tools · 7m 12s
	✓ gubbi:extraction-deep (Claude Code) claude-deepseek-pro 131.4k tok · 97 tools · 10m 18s
	✓ gubbi:python Sonnet 4.6 73.1k tok · 27 tools · 5m 19s
	✓ common:deep claude-deepseek-pro 134.7k tok · 56 tools · 5m 45s
	✓ common:security claude-codex-pro 21.5k tok · 9 tools · 1m 16s
	✓ testbench:deep claude-deepseek-pro 103.8k tok · 79 tools · 6m 16s
	✓ infra:devops claude-codex-pro 67.9k tok · 33 tools · 8m 46s
	✓ infra:security claude-codex-pro 112.4k tok · 40 tools · 5m 51s
	✓ infra:architecture claude-deepseek-pro 94k tok · 42 tools · 5m 57s

Obrázek 5: Postup práce dynamického workflow v Claude Code: vlevo fáze plánu se stavem, vpravo rozvinutá fáze *CodeReview* se šestnácti agenty, u každého model, spotřebované tokeny, počet nástrojů a doba běhu. Screenshot pochází z komunitního příspěvku; rozhraní i jeho chování popisuje dokumentace (34).



```
Antigravity CLI 1.0.0
Gemini 3.5 Flash

> can you start 3 subagents to research how to build a data web app? E.g. a UI agent,
a data agent and a code quality analyst

▸ Thought for 2s, 852 tokens
Initiating Project Decomposition

● DefineSubagent(Subagent "ui_researcher" defined successfully. It can now be...)
● DefineSubagent(Subagent "data_researcher" defined successfully. It can now ...)
● DefineSubagent(Subagent "code_quality_analyst" defined successfully. It can...)
(ctrl+o to expand)

▸ Thought for 2s, 634 tokens
Defining Subagents Concurrently

● Agent(ui_researcher: UI/UX Researcher)(Please research and provide a comprehensive
report on building a premium-grade U...)

I have successfully defined and launched 3 specialized subagents to research
different aspects of building a high-quality data web application:

1. UI/UX Researcher ( ui_researcher )
```

Obrázek 6: Rozhraní Google Antigravity CLI, tedy harnessu agenta běžícího v terminálu: koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem (38).

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdíleného stavu a integračních kontrol. Orchestrace je však nezbytná právě pro rozsáhlé úlohy, které se do jednoho kontextového okna nevejdou. Spoléhat se v takovém případě na kompakci by vedlo k tomu, že se ztrácí kontext, na kterém celé zadání stojí, a agent by navíc nebyl schopen pracovat na jednotlivých částech, které na sebe bezprostředně navazují.

2.3 Softwareová továrna

Pojem softwareová továrna je starší než generativní umělá inteligence o přibližně padesát let. Zavedl jej Robert Bemer z General Electric ve svém návrhu strojově řízeného výrobního prostředí, který představil na konferenci NATO o softwarovém inženýrství v Garmischi v říjnu 1968. Zpráva z konference jeho formulaci cituje: „By utilizing the machine-controlled production environment, or software factory. Program construction, checkout and usage are done entirely within this environment using the tools it contains. Ideally it should be impossible to produce programs exterior to this environment.“ (39) Konference se zabývala především softwarovou krizí, nikoli zavedením tohoto pojmu — sama o prvním užití neuvádí a pojem pouze připisuje Bemerovi (39). Že pojem zavedl a vymezil jeho povahu, uvádí jeho biografie (40). O jeho návrhu se na konferenci mluvilo jako o tom nejambicióznějším přístupu, jaký byl představen (41). Přitom se shodlo, že automatizace výroby sama o sobě úsudek neruší. „Human monitoring of production is very adaptive — the automated system may disguise some of what is happening“, upozorňoval Fraser (39). „If you don't know what you're doing in producing software, then automating the system can be dangerous“, dodal Ross (39). Dodal McIlroy: „It would be immoral for programmers to automate everybody but themselves.“ (39) O devět let později Bemer sám upřesnil, co továrna kromě nástrojů potřebuje: „One does not build a successful factory with just tools and environment. The workers must be trained, and the assembly methodology (e.g., drawings, parts lists, inventory) must be in place.“ (42)

Agenti posunuli hranici, za kterou továrna končila. Zatímco klasická softwareová továrna automatizovala kroky, jejichž postup byl předem dán, agent vykoná i krok, jehož postup předem dán není: zadání přeformuluje, plán sestaví, chybu, kterou sám způsobí, opraví a rozhodne, zda je na řadě další nástroj. Automatizovatelná část se tím posouvá z provádění na úsudek, zatímco člověk zůstává tam, kde byl v každé dosavadní továrně — u určení, co je správné a co smí do výroby vstoupit.

Právě proto se pojem vrací do užívání. Současné popisy této architektury ji označují právě jako software factory: systém, v němž agenti nepřetržitě nacházejí práci, plánují ji, implementují, revidují a předávají výsledek člověku (43; 44). Popisovaný průběh — událost spustí plánování, implementace probíhá na izolované větvi, revizi provádí oddělený krok, sloučení zůstává lidskou bránou — odpovídá tomu, co popisují i systémy postavené mimo tuto práci.

Odlišnost je přesto jednoznačná. Společnost Guild, která takový systém provozuje a sama jej takto označuje, uvádí, že 34 % sloučených pull requestů vzniká autonomně, 56 % oprav kódu pochází z její factory a 91 % jejich sloučených pull requestů nevyžaduje zásah inženýra

(45). Jde ovšem o vlastní měření, nikoli o nezávislé ověření, a oproti popsanému stavu má architektura jednu zásadní odlišnost: plánování, implementaci a revizi u nich zajišťují oddělení agenti, zatímco v této práci jde o tytéž role v jediném harnessu, lišícím se pouze zadáním. Tvrzení o tom, že se tento průběh dnes stává běžnou praxí, lze opřít o průmyslové zprávy o šíření agentů do výroby (46).

Z toho plyne i upřesnění, co práce sama přidává. Popsaná pipeline nevymýšlí softwareovou továrnu znovu; naopak na záměrně minimálním provedení ukazuje, že její provozní podstaty lze dosáhnout volenou sadou cizích nástrojů. Tvrzení, že právě tato část je vlastním přínosem práce, je tedy formulováno jako demonstrace, nikoli jako novost.

PRAKTICKÁ ČÁST

DarkFactory

3 Praktická část

3.1 Metodika

Praktická část implementuje záměrně jednoduchou produkční pipeline, v níž je vývojovým prostředím přímo GitHub a harness produkční coding agent. Cílem je ukázat, jak lze spojit události GitHubu, automatizované plánování, izolovanou práci v kontejneru a lidskou integraci do jednoho opakovatelného procesu.

Záznam je určen jedinou věcí: revizí repozitáře, z níž byl pořízen, a všechny popisy níže vycházejí z této revize, nikoli z jejího pozdějšího vývoje (15). Bez tohoto určení by nebylo možné ověřit, zda popis odpovídá kódu, a právě na tomto místě se ukázalo, že několik dřívějších tvrzení o popsáném průběhu revizi neodpovídalo.

Předmětem analýzy jsou čtyři navazující vrstvy:

1. *GitHub jako zdroj pravdy*: issue a jeho komentáře uchovávají požadavek, schválení, plán a zpětnou vazbu; větev, commit a pull request uchovávají změnu a její průběžnou revizi (47; 48).
2. *GitHub Actions jako výpočetní prostředí*: jednotlivé události spouštějí krátké workflow, které provede checkout pracovní kopie repozitáře, sestaví obraz, spustí agenta a následnou integraci provede.
3. *Python a Docker jako izolační vrstva*: workflow předá událost a pracovní strom pythonovskému runneru v kontejneru; runner volá harness a předává mu nástroje, přihlašovací údaje a stav úlohy.
4. *Review smyčka jako podmínka integrace*: automatická revize diffu, opravy a další revize se opakují, dokud poslední nález nezmizí, v popsané revizi však nejvýše třikrát; teprve poté je pull request předán člověku.

Metodika sama se řídí tvrzením práce. pythonovský runner, o kterém tato část pojednává, byl napsán co nejjednodušeji pomocí produkčních coding agentů, tedy právě těm nástrojům, které následně popisuje. Cílem nebyla stavba co nejucelenějšího systému, ale co nejmenší uzavřená smyčka, na níž lze ukázat, jak se navzájem ovlivňují agent, harness a lidský recenzent. Pozornost je proto soustředěna na tuto interakci a na její členění, nikoli na širší nabízených funkcí.

Zjednodušení je záměrné a má svou cenu. Systém bez vlastního stavového serveru, bez databáze a bez trvalého běžícího procesu je na první pohled méně schopný než plnohodnotná platforma, oproti níž je však reprodukovatelný, dohledatelný a jehož každé rozhodnutí zůstává vidět v issue, commitu nebo průběhu kontroly. Zároveň umožňuje zvyšovat složitost postupně: každý nový prvek pipeline byl přidán až poté, co bylo na konkrétní úloze zřejmé,

že současná podoba nestačí. Samotná pipeline se tak stala nástrojem, kterým byla psána i další její část.

Druhy zdrojů a to, z čeho se v práci čerpá, jsou uvedeny v úvodu. Zde zbývá jedno pravidlo: citovat lze jen text, který autor otevřel.

3.1.1 Tři kritéria

Sloveso výzkumné otázky je *musí* a je míněno podmíněně: nutné jsou ty principy, které samotný model bez zásady okolí neposkytne. Aby to nebylo tvrzení bez měřítka, převádí práce tuto podmíněnou nutnost na trojici kritérií, podle níž lze konkrétní systém posoudit. Jde o *operacionalizaci této práce*, nikoli o převzatou taxonomii: pět oborů agentického inženýrství je slovník, který používají zdroje, ale kritéria nejsou jeho podmnožinou ani překladem.

Stav leží mimo model a je zjiřitelný. Kritérium vyžaduje pojmenovat artefakt, úložiště, kde leží, a osobu, která do něj může nahlédnout, aniž by se musel ptát modelu. Stav uložený v konverzaci je také stav, ale jeho jediným čtenářem je model, který v něm pracuje.

Brány jsou výslovné a druhy jejich rozhodnutí se nerozplývají. Kritérium vyžaduje, aby brána existovala, věděla, jaké rozhodnutí činí, a označila své rozhodnutí za druhové. Druhů jsou tři: deterministická kontrola, jejíž výsledek lze přepočítat nebo znovu spustit bez modelu; modelem hodnocená brána, která je dotazem na jiný model, a je proto stejně pravděpodobnostní jako to, co posuzuje; a lidská brána. Běh nesmí projít bránou jednoho druhu prostředky jiného: modelem hodnocená brána nenahrazí deterministickou kontrolu, protože neporovnává výsledek s tím, co se dá přepočítat, a lidská brána bránu předchozí nenahrazuje, nýbrž ji schvaluje. Toto rozdělení není vynalezeno zde; dokumentace, která popisuje vládnutí běhu, uvádí bránu, která „can run a script for deterministic checks or a prompt for model-evaluated ones“ (37), a druhou z nich odhazuje právě proto, že je pravděpodobnostní.

Člověk rozhoduje, co vstoupí do produkce. Kritérium je formulováno jako nepodléhající vyjednávání: v popsaném systému musí existovat pojmenovaný okamžik, v němž rozhodne určený člověk o určené věci, a nesmí existovat žádná jiná cesta, která by ke stejnému bodu vedla. Podmínka „určený“ je ostrá: musí jít o osobu, kterou lze pojmenovat, nikoli o kohokoli, kdo v okamžiku klikne. Kritérium je jediné, které práce neváže na výsledek měření a neoslabuje, protože systém, který je nesplňuje, neprovádí podle ní inženýrskou práci, ale automatizovanou výrobu změn bez odpovědného člověka — a to je přesně ta věc, kterou konference v Garmischi odmítla bez důvodu (39).

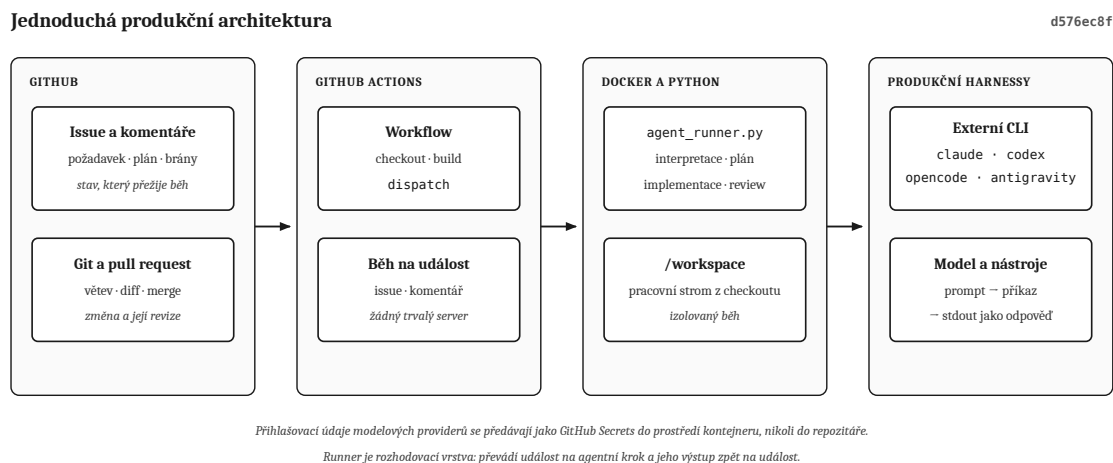
3.1.2 Otevřená otázka: složení vrstev

Systém popsaný v této práci má nad sebou dvě vrstvy harnessu. Vnější je runner s branami a uloženým stavem, vnitřní je produkční CLI, které modelové kroky vykonává. Během jednoho běhu se tedy model setkává se dvěma sadami pravidel současně. Které z nich má převážít a co stojí vnější vrstva, na to teoretická část neodpovídá.

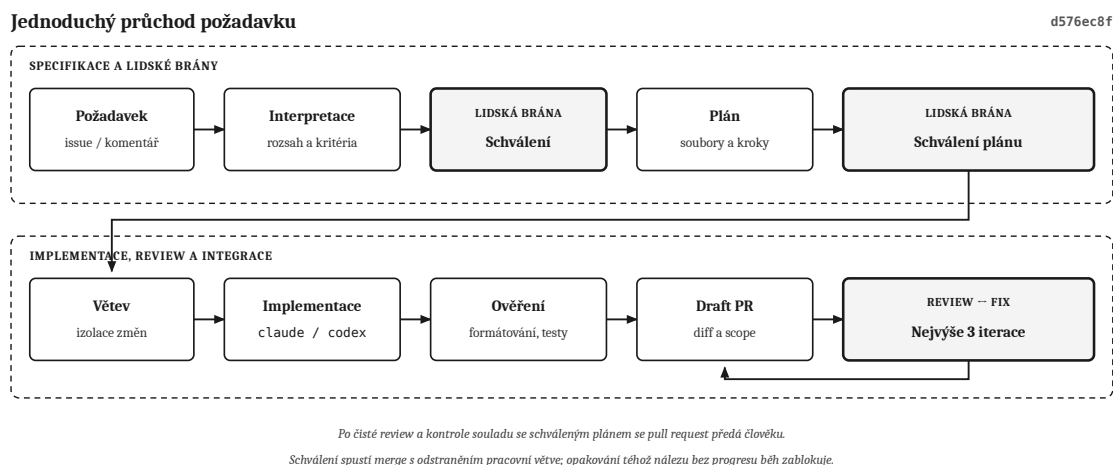
Část odpovědi je přesto veřejně dostupná a práce ji nechává stát, místo aby ji doplnila odhadem. Dokumentace, která popisuje vrstvení běhů, rozlišuje předání práce, při němž zůstává jediný cyklus, od volání agenta jako nástroje, při němž vzniká vnořený běh s vlastním cyklem, limitem iterací a žádostmi o schválení. Není tím řečeno ani to, kterému pravidlu se model podřizuje, ani jak vysoký je náklad: „the nested turns do not increment the outer run’s turn counter“ (31), takže účetnictví vnější vrstvy práci vnořeného běhu nevidí. Zda je takový náklad v popsaném systému významný, zjištěno nebylo. Záznam opakovatelnosti výše na otázku neodpovídá, protože z něj nelze vyčíst, čím se spouštěná smyčka liší od jiné, a práce to nezaměňuje za odpověď na otázku o konvencích. Otázka zůstává otevřená a je evidována jako omezení, nikoli jako vyřešený bod.

3.2 Architektura produkčního běhu

Pro základní průchod nepotřebuje DarkFactory nic, co by muselo běžet trvale na vlastním počítači. Všechno, co pipeline potřebuje, už existuje: GitHub slouží jako rozhraní i jako trvalý stavový systém a každá práce agenta probíhá jako izolovaný běh v GitHub Actions. Členění do čtyř vrstev znázorňuje Obrázek 7 a pořadí jednotlivých kroků Obrázek 8.



Obrázek 7: Architektura: GitHub poskytuje události a vývojový stav, GitHub Actions výpočet, Docker odděluje běh a pythonovský runner převádí událost na agentní krok. Produkční harnessy zajišťují model, nástroje a pozorování; registr jich má osm, obrázek ukazuje čtyři (15).



Obrázek 8: Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka se opakuje, dokud nálezu nezmizí, v této revizi nejvýše třikrát (15).

Workflow `agent.yml` reaguje na otevření issue, nový komentář, review komentář, `repository_dispatch` nebo ruční spuštění. Podmínka na úrovni jobu ověřuje, zda je agent pro repozitář povolen, a filtruje automatické komentáře, aby vlastní výstup pipeline nevytvářel nové události. Workflow pak provede checkout cílového repozitáře, sestaví obraz

podle `docker/Dockerfile.agent` a spustí příkaz `dispatch` v kontejneru. Obraz je sestaven z jedné definice, takže každý spotřebitel běží stejný runner, zatímco repozitář, na němž se pracuje, zůstává checkoutem volajícího.

pythonovský runner není náhradou harnessu. Je rozhodovací a integrační vrstvou, která převádí událost GitHubu na konkrétní agentní krok, zpracovává jeho výstup a vyvolává další událost. Modelové kroky jsou přitom stále prováděny harnessem nad explicitně předaným pracovním stromem (15).

Každý harness je tak definován deklarativně: binár, způsob, jak se z promptu sestaví příkazová řádka, a způsob přihlášení. Přidání harnessu je tedy změna dat, nikoli kódu (15). V popsané revizi je v registru osm harnessů a pořadí, v němž se zkoušejí, je konfigurovatelné; harness, jehož binár v obrazu chybí, se přeskočí, takže jediný obraz unese jen podmnožinu nástrojů, která je v něm nainstalována. Tvzení, že struktura procesu nezávisí na volbě nástroje, je proto v této revizi doslova pravdivé: strukturu popisuje registr, ne kód.

Přihlašovací údaje se neukládají do repozitáře. GitHub App nebo jiný autorizovaný token se používá pro checkout, issue, pull requesty a push; přihlašovací údaje modelových providerů jsou předány workflow jako GitHub Secrets a následně prostředím kontejneru. Tím pipeline odděluje své automatizační oprávnění od přihlašovacího materiálu agenta a umožňuje změnit poskytovatele bez změny pracovního stromu (15).

3.3 Interpretace a plánování požadavku

Výchozím bodem je issue s formulovaným požadavkem. Požadavek se však nevytváří jako volný text: repozitář nabízí předlohu, která vyžaduje vyplnit několik polí, a právě ta určují, co pipeline dostane. Tabulka 1 uvádí tři povinná pole předlohy a skutečný požadavek, který předlohu vyplněnou nemá.

Pole předlohy issue

Jak
to
vy-
pa-
dá
v
is-
sue
727

Verbatim User Request povinné

re-
s-
to-
re
the
agent
con-
tai-
ner
build
from
the
che-
c-
ked-
-in
Bun
lock
ow-
ner

Area / Component povinné

 area:ci

Request Type povinné

 bug

Problem — `Dockerfile.agent` kopíruje kořenový `bun.lock`, ale repozitář má jen `harness/bun.lock`; běh agenta tak padá ještě před spuštěním. Viz `### Contract` v issue.

Tabulka 1: Předloha požadavku a nejstarší požadavek projektu, issue 727 z 20. září 2026. Předloha má šest polí, z nichž tři povinná. Skutečný požadavek však předlohu nevyplnil, ale napsal volně, pod nadpisy `Problem` a `Contract`; tabulka proto ukazuje tři povinná pole předlohy a to, jak požadavek ve skutečnosti vypadá. Runner předává modelu titulek i tělo issue, takže právě to rozhoduje, co se do zadání dostane (15).

Příklad je záměrně malý a měřitelný: rozsah se vejde do jednoho kontextového okna a podmínky dokončení lze přepsat na testy. To nejsou náročky pipeline, ale podmínky, za

kterých je nasazení vůbec smysluplné — a právě proto je předloha součástí popisu. Je tu ale i věc, kterou předloha neumí: přijaté akceptační podmínky jsou text, který vstupuje do kontextu, a nikdo je v popsané revizi nepřevádí na spustitelnou kontrolu. Podmínka, kterou nelze spustit, se při prokazování chová stejně jako podmínka, která neplatí.

Runner načte jeho titulek a text, vyžádá si od modelu interpretaci a zapíše výsledek jako komentář. Komentář má oddělovat doslovné shrnutí požadavku, architektonický rozsah a návrh verifikace. Tím se z chatové odpovědi stane zkontrolovatelný návrh, který lze před dalším během přijmout nebo opravit. Na rozdíl od prostého chatu je tu interpretace artefakt, který přežije běh a dá se k němu vrátit.

Po schválení interpretace je workflow spuštěno znovu a runner založí *dítě* issue s názvem začínajícím na `Plan:`, nativně propojené s původním issue. Model dostane schválený požadavek a sestaví implementační plán s očekávanými změnami, soubory nebo oblastmi repozitáře a kroky ověření. Plán se opět zobrazí v issue a schvaluje se samostatně. Schválení je tak explicitní bránou: samotná schopnost agenta plán vytvořit neznamená oprávnění měnit kód. Obě brány před vznikem větve jsou lidské, a protože jsou oddělené, lze schválit porozumění zadání a odmítnout plán, který z něj vychází.

Zpětná vazba člověka není součástí nového vývoje od začátku. Komentář se změnou nebo odmítnutím se předá zpět interpretaci nebo plánování, takže se opravuje rozhodnutí před vytvořením pracovní větve. Tento jednoduchý model odděluje porozumění zadání, plánování a vlastní implementaci bez potřeby složitého grafového orchestrátoru (15), což je volba, již literatura připouští jako legitimní: paralelní práce je užitečná tam, kde na sobě úlohy nezávisí, a jinde je to jen zdroj sporu (33).

3.4 Implementace a automatická revize

Po schválení plánu runner vytvoří nebo načte pracovní větev odvozenou z výchozí větve. Implementační instrukce obsahuje schválený plán, omezení na jeho rozsah a pravidla pro testy a dokumentaci. Harness následně může procházet repozitář, upravovat soubory a používat nástroje nad `/workspace`; změny zůstávají izolované mimo výchozí větev.

Po implementaci runner spustí dostupné formátovací nástroje a deklarované testovací sady. Při neúspěchu předá výstup kontroly agentnímu kroku `fix`, který má opravit chybu bez opuštění schváleného rozsahu. Testovací sady se přitom spouštějí pouze jednou a po opravě už ne: kód se zformátuje, zakomituje a odešle dál, i kdyby opravená verze stále neprocházela. Opakovatelná kontrola tedy rozliší dobrý a špatný stav, ale není překážkou — změna s neprocházejícími testy se dostane do draft pull requestu stejně jako změna procházející. Následně runner vytvoří commit, odešle větev a otevře draft pull request, čímž se změna oddělí od svého posouzení: implementace může být dokončena, ale pull request ještě není připraven k merge (15).

Na draft pull requestu začíná automatická review smyčka. Každá iterace načte aktuální větev a diff a předá je modelové revizi, zatímco průběžná integrace znovu spustí tytéž nástroje. Nalezený problém runner zveřejní a spustí nový běh s fází opravy, která změní větev a spustí další review. Nález vyžadující zásah mimo plán se neodmítne: runner jej zapíše jako Plan Deviation s odůvodněním na původním issue a plán tím doplní. Schválený plán tedy není neměnný, ale každá jeho změna je zapsaná a odůvodněná. Smyčka je ohraničena nejvýše třemi iteracemi a oprava, která větev nezmění, ji ukončí dřív. Po vyčerpání tří iterací však smyčka skončí bez jakéhokoli verdiktu a bez komentáře, pull request zůstane ve stavu draft a nikdo zvenčí nepozná, zda review prošlo nebo se jen zastavilo. Jediným stavem, který v této revizi znamená zablokování, je vyčerpaná kvóta — tehdy se zapíše kontrolní bod, stav na projektové tabuli a komentář s výzvou k obnovení (15).

Tabulka 1 ukazuje požadavek podaný právě kvůli této meze: žádá, aby se běh po třech opakováních stejného nálezu *zastavil a napsal to výslovně do issue*. Popsaná revize to neumí, ať žádá o třetí opakování, ačkoli je podmínka zastavení v té dokumentaci uvedena výslovně (32).

Po čisté review ještě proběhne kontrola souladu výsledného diffu se schváleným plánem. Ani ta není porovnáním sad souborů: je to druhý dotaz modelu, tentokrát na shodu s textem plánu. Teprve když oba dotazy vyjdou bez nálezu, je draft pull request označen jako připravený k lidské revizi. Je důležité, že automatická revize může skončit bez nálezu, aniž

by zaručila, že implementace odpovídá zadání, a posuzuje přitom plán, jehož text napsal tentýž model. Dodavatelé tento typ hodnotitele popisují stejně omezeně (37), a je proto s první branou rovnocenný, ne její náhradou.

3.5 Zpětná vazba, schválení a úklid

Po otevření pro lidskou revizi může člověk použít schvalovací workflow nebo zpětnou vazbu na pull requestu. Změnový požadavek zachycuje runner jako opravný běh na stejné větvi. Agent obdrží plán, konkrétní zpětnou vazbu a aktuální kontext větve, provede úpravu, uloží ji jako commit a znovu spustí automatickou review smyčku. Tím se zpětná vazba nevrací do schváleného plánu ani nevyžaduje nový počátek celého požadavku; zachovává se pouze řetězec revizí na jednom pull requestu (15).

Schválení pull requestu zpracovává samostatný workflow. Ten nejprve ověří, kdo akci provedl, a ověření je užší, než by se z krátkého popisu zdálo: aktér se porovnává se seznamem dvou pevně zapsaných účtů, vlastního repozitáře a jednoho dalšího. Neověřuje se tedy autor issue ani úroveň oprávnění, ale členství v tomto seznamu (15). Teprve pak workflow převede draft na ready stav, zkontroluje požadované schválení a zapne merge; při úspěchu použije `--delete-branch`, takže se pracovní větev po sloučení odstraní. Integrace tedy nekončí pouhým otevřením pull requestu: zahrnuje přechod do ready, splnění ochrany větve, merge a bezpečné odstranění větve (15; 48).

Je tu třeba přiznat věc, která třetí kritérium metodické části oslabuje. Jestliže ochrana větve stále vyžaduje schválení, workflow podá *zástupné* schválení jménem jiného účtu, aby merge neuvázl (15). Podmínkou je druhý přihlašovací údaj; kde není, pipeline čeká na skutečné schválení člověka. V konfiguraci, v níž je nastaven, je však schválení, které by mělo být lidským rozhodnutím, vydáno strojem. Třetí kritérium je proto plněno jako *bod*, na kterém člověk rozhodne, a ne jako *záruka*, že rozhodne on; rozdíl je malý v textu a velký v tom, co z něj plyne, a práce ho záměrně nerozšiřuje.

Tato architektura je záměrně jednoduchá. GitHub poskytuje události, schválení, uložení změn a průběh kontroly, GitHub Actions výpočet, Docker odděluje běh, Python koordinuje a harness vykonává agentní práci. Složitější služby, trvalý stavový server a více souběžných agentů jsou záměrně mimo základní návrh. Výhodou je snadná reprodukovatelnost a viditelnost každého rozhodnutí; nevýhodou je zároveň závislost na dostupnosti GitHubu a na kvalitě promptu, modelu a pravidel repozitáře, kterou samotná automatizace neodstraňuje.

Směr vývoje, který práce záměrně nepopisuje, je vymezen v Kapitola 1.2. Zde je tedy přesnější dodat, kde popsán průchod končí: je popsán stav, v němž pythonovský runner volal cizí produkční CLI (15). Popsaná architektura je tudíž mezníkem, nikoli vyvrcholením, a její popis nenahrazuje posouzení této meze, které patří do Kapitola 4.2. Projekt (49) se od tohoto

stavu posunul dál; kam se posunul a proč, je předmětem práce následující, a zde to záměrně není rozvedeno.

4 Závěr

4.1 Zjištění

První zjištění se týká volby vývojového prostředí. Požadavek, jeho schválení, plán, zpětná vazba i výsledná revize zůstávají v GitHubu, takže jednotlivé běhy nemusí sdílet vlastní databázi ani trvalý proces, jak popisuje Kapitola 3.2. Plán je samostatné issue propojené s původní žádostí, takže se k němu dá vrátit a porovnat s výsledným diffem (15).

Druhé zjištění potvrzuje oddělení lidského rozhodnutí od modelového návrhu. Issue nejprve vyvolá interpretaci, následně plán a teprve po schválení obou kroků se vytvoří pracovní větve a spustí implementace. Tím pipeline nevytváří změny v hlavní větvi pouze na základě samotného návrhu agenta (15).

Třetí zjištění se týká průběhu revize a jeho pořadí. Implementace je nejprve odevzdána jako draft pull request. Automatická kontrola nejprve předá diff modelové review a teprve poté, jakmile je čisté, porovná změněné soubory se schváleným plánem. Nalezené problémy spouštějí samostatný běh opravy, po němž následuje nová kontrola. Teprve čistá review a kontrola souladu s plánem otevrou pull request pro lidskou revizi (15).

Čtvrté zjištění se týká zpětné vazby a integrace. Změnový požadavek na pull requestu spouští opravu na stejné větvi a po pushi znovu vstupuje do review smyčky. Schválení pull requestu naopak spouští merge s odstraněním větve, takže pipeline nekončí automatickým vytvořením kódu, ale přechází do explicitního lidského schválení a následné integrace. Kdo schválení vykoná, je v této revizi určeno jen dvojčlenným seznamem účtů, a je-li ochrana větve stále žádající, může potřebné schválení doplnit workflow jménem jiného účtu (15).

Páté zjištění je záporné a práce ho ponechává. Tři předpoklady, na nichž popis průběhu stál, v této revizi neplatí. Revize se po vyčerpání tří iterací zastaví bez zaznamenaného verdiktu, takže nerozliší průchod od zastavení; zablokovaný stav zde neznamená opakovaný nález, ale vyčerpanou kvótu; a testy se před sloučením nespouštějí znovou, takže změna, která neprošla, na pull request dorazí. Podmínka zastavení, která první z těchto mezer vyřešila, je v dokumentaci cílové smyčky popsána výslovně (32); její vyplnění je věcí následné práce, ale zjištění, že chybí, do výsledků patří (15).

Zjištění je zároveň možno převést na kritéria vymezená v metodické části. Prvnímu odpovídá stav, který práce popisuje výše: dvě artefakty mimo konverzaci, k nimž lze později vrátit. Druhému odpovídá to, že každá brána své rozhodnutí označuje za druhové, ačkoli diskuse upozorňuje, že obě brány před sloučením jsou dotazy téhož druhu. Třetímu odpovídá pop-

saná konfigurace jen podmíněně: pojmenovaný okamžik, v němž rozhodne určený člověk, existuje, ale ve variantě se zástupným schválením jej může vyplnit stroj, a znění kritéria právě nepřítomnost jiné cesty požaduje. První dvě kritéria tedy popsaná konfigurace plní a třetí plní podmíněně, za podmínky, kterou sama zaznamenává (15).

4.2 Diskuse

Zjištění lze interpretovat jako konkrétní podobu agentického inženýrství: praktická autonomie není vlastností samotného modelu, ale výsledkem návrhu prostředí, v němž model pracuje. GitHub poskytuje trvalý kontext a lidskou odpovědnost, Python určuje přechody mezi kroky, Docker omezuje běhové prostředí a harness propojuje model s nástroji a pozorováními (15). Tato dělba odpovědnosti odpovídá tomu, jak obor takové systémy popisuje (8; 9).

Review smyčka ukazuje výhodu a zároveň omezení průběžné kontroly. Testy a formátovací nástroje poskytují zpětnou vazbu, kterou lze opakovat, ale ani druhá brána — kontrola souladu s plánem — deterministická není: jde o druhý dotaz modelu, a je tedy stejně pravděpodobnostní jako review, které jí předchází. Opakované iterace zvyšují počet příležitostí k nálezům, nikoli záruku konečné správnosti. Zastavení po vyčerpání iterací je ochranou před nekonečným během, ale bez zaznamenaného verdiktu není ani touto ochranou úplně: kdo se později ptá na stav, nemá v issue co číst (15).

Záporné zjištění ukázalo i druhou věc. Opakovatelná kontrola v této revizi rozliší dobrý a špatný stav, ale není překážkou: testy běží jednou a po opravě už ne, takže změna, která neprošla, se do pull requestu dostane stejně jako změna, která prošla. Rozlišení mezi kontrolou, která brání, a kontrolou, která pouze hlásí, je tedy neúplné, a je to vlastnost konfigurace, nikoli principu. Dodavatelé, kteří takovou bránu prodávají, popisují její omezení shodně: hodnotitel nevolá nástroje a posuzuje jen to, co pracující model sám ukázal (37). Samoshodnocení agentů selhává určeným směrem (8). Rozdělení práce od hodnocení tedy shovívavost neodstraňuje samo o sobě; odstraňuje ji až to, že kontrolu, kterou by bylo možné zopakovat, zajišťuje návratový kód místo názoru.

Jednoduchost návrhu má také ekonomickou a epistemickou cenu. Nevyžaduje žádnou vlastní infrastrukturu a celý průběh je dohledatelný v issue, commitech, kontrolách a pull requestu. Na druhé straně pipeline závisí na dostupnosti GitHubu, kvalitě issue, schopnosti modelu pracovat s nástroji a pravidlech, která definují akceptační podmínky. Jednoduchá architektura tedy usnadňuje reprodukci postupu, ale nenahrazuje odpovědnost člověka za architekturu, bezpečnost a přijetí výsledku (15).

Opakovatelná a pravděpodobnostní pozorování se proto doplňují, ale nenahrazují. Formátovací nástroje a testy poskytují pozorování, které se dá opakovat; review, kontrola souladu s plánem, plánování a opravy interpretují zadání a neočekávané nálezy. Z toho plyne, že obě brány před sloučením nejsou nezávislé: jsou to dva dotazy téhož druhu, a druhý z nich navíc

vychází z plánu, jehož text sám napsal model. Tato kombinace odpovídá cíli agentického inženýrství zvýšit užitečnost delegované práce při zachování explicitních bran, ale její skutečná kvalita zůstává závislá na datech, promptu, modelu a konkrétním repozitáři (15).

4.2.1 Omezení výzkumu

Praktická část má omezení, která je třeba pojmenovat, protože se dotýkají platnosti závěrů. Spočívají ve dvou vrstvách.

První je vlastnictví. Agentní smyčku v popsané realizaci autor nenavrhuje: sestavuje ji cizí harness, která spravuje kontext, volá nástroje a vyřizuje oprávnění. Co autor navrhuje, je vrstva nad ní, tedy runner, který rozhoduje, kdy se smyčka spustí, co smí agent změnit a kde práce skončí. Odpověď na výzkumnou otázku je tím užší, než kdyby smyčka byla implementována od začátku, a je takto i formulována.

Druhé je rozsah. Zkoumaný stav je záměrně jen počáteční: runner volá cizí harnessy a všechny agentní kroky vede přes ně. Jde tedy o míru autonomie dosažitelnou touto konkrétní konfigurací, nikoli o míru dosažitelnou s vlastní smyčkou, a tím záměrně neřeší otázku, kterou řeší práce následující. Tvzení práce se proto vztahuje k této konfiguraci a je opřeno o konkrétní revizi (15).

Z prvního omezení plyne i obrana závěrů. Harness je v popsané architektuře zaměnitelnou závislostí, stejně jako databáze nebo runner. Co práce především ukazuje, není volba konkrétního CLI, ale struktura procesu: že požadavek musí projít interpretací, plánem a dvěma lidskými branami dříve, než vznikne větev, a že změna se přebírá jako konkrétní revize na jednom pull requestu, dokud nesplní obě brány. Tato struktura je vůči tomu, kdo vykonává modelové kroky, invariantní. Právě to je obsahem agentického inženýrství ve smyslu vymezeném v teoretické části: jde o vlastnost této práce, neboť popsaná struktura je záměrem autora a nezávisí na tom, kdo vykonává modelové kroky. To však není obecná zákonitost, a závěr práce proto přenositelnost struktury na libovolný nástroj neprokazuje; naopak, tam, kde se konvence přenášejí z jednoho typu běhu do druhého, je výsledek opačný (13). Hypotéza se potvrzuje: strukturu, kterou práci řídí, autor navrhuje sám, zatímco úsudek o tom, co je v jednotlivém kroku správné, přebírá od smyčky, kterou nevlastní.

4.3 Shrnutí

Jazykový model sám o sobě pouze generuje výstup v závislosti na vstupním kontextu. Agent z něj vzniká až propojením s nástroji, prostředím, stavem a pozorováním, které zajišťuje harness (8; 23). Ani toto propojení však pro účinné nasazení ve vývoji softwaru nestačí bez postupů agentického inženýrství, které vymezují rozsah autonomie, určují body lidského rozhodnutí a zajišťují ověřování výsledků. Na výzkumnou otázku, které principy musí agentický systém splnit, aby vykonával inženýrskou práci, odpovídá tato práce třemi principy: stav mimo model a zjistitelný, brány s druhy rozhodnutí, které se nerozplývají, a rozhodnutí člověka o tom, co vstoupí do produkce. Podmíněná nutnost znamená, že nutné je to, dokud model sám danou práci spolehlivě nevykoná; s rostoucí schopností modelů se tato hranice posouvá, ale ne mizí.

Implementace pipeline DarkFactory ukázala jednoduchou produkční realizaci těchto postupů. GitHub slouží jako vývojové prostředí a místo trvalého stavu, GitHub Actions jako výpočetní prostředí, Dockerový kontejner jako izolace a produkční harness pro modelovou práci s nástroji. Požadavek prochází interpretací a plánováním, implementace probíhá na samostatné větvi a změna je předána přes draft pull request a opakovací smyčku review–fix. Po čisté review a kontrole souladu s plánem následuje lidské schválení, merge a odstranění větve.

Úloha vývojáře se v tomto uspořádání posouvá od rutinního provádění kódu k formulaci zadání, schvalování plánu, definování akceptačních podmínek a rozhodování, zda je výsledek připraven k integraci. Práce tedy nepopisuje plnou autonomii agenta, ale kontrolované předání vybrané části softwarového inženýrství.

Zjištění zároveň ukazují meze této realizace. Modelové review může být opakované, ale není deterministickou zárukou správnosti. Review smyčka se po vyčerpání tří iterací zastaví bez zaznamenaného verdiktu, takže průchod od zastavení rozlišit není; zablokovaný stav zde znamená vyčerpanou kvótu, nikoli opakovaný nález; a opakovatelná kontrola před sloučením běží, ale změně s neprocházejícími testy zabrání jen tehdy, když ji zopakuje, což popsaná revize nečiní.

Cíle práce se podařilo naplnit v rozsahu, který si sama vymezila. Hypotéza se potvrdila: praktická autonomie je vlastností návrhu systému, který práci řídí, a nikoli vlastností modelu, který v něm pracuje. Strukturu, která práce popisuje, autor navrhuje sám; úsudek o tom, co je v jednotlivém kroku správné, však přebírá od smyčky, kterou nevlastní, a právě v tomto rozdílu leží hranice, za kterou by pomohlo vlastnit i tuto smyčku.

Pro další výzkum z toho vyplývají tři směry. První je vlastní harness: popsat architekturu, kterou si tato práce záměrně neřešila, a porovnat její výsledky s výsledky cizích nástrojů na stejných úlohách. Druhý je měření: dosavadní zjištění jsou kvalitativní, chybí jim to, co by umožnilo říct, o kolik se cyklus zkrátil a kde přesně review smyčka nachází nejvíce nálezů, a prvním krokem je nechat rozlišit čistou recenzi od zastavené smyčky a nechat opakovanou kontrolu běžet znovu, protože dnes hlásí, ale nebrání. Třetí je otázka, která zůstala v metodické části otevřená: když vnější vrstva je sama o sobě harnessem, setkává se model během jednoho běhu se dvěma soubory konvencí. Práce neprovedla měření, které by ukázalo, kterých následuje a co vnější vrstva stojí Kapitola 3.1.2.

V praxi má práce přímý dopad v podobě jednoduchého pravidla: delegovat agentovi až to, co lze zkontrolovat, a ponechat mu vlastní smyčku jen tam, kde je výhodná. Kdo takovou pipeline staví, nemusí začínat vlastním harnessem; začíná stavem uloženým mimo model, dvěma branami před vznikem větve a oddělenou kontrolou výstupu. A ještě jedno, které z popsaného systému vyplynulo a které je stejně důležité jako tyto tři: je-li brána, která má být lidská, v jedné konfiguraci schopna vyplnit stroj, pak její jméno neznamena nic, pokud u ní není zapsáno, kdo ji vykonává.

Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. GITHUB. *Asking GitHub Copilot questions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide>
3. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
4. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-codex/>
5. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-the-codex-app/>
6. TRIVEDY, Vivek. *The Anatomy of an Agent Harness* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>. LangChain, 10. března 2026
7. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://www.gradually.ai/en/how-many-people-use-ai/>. Published 2026-08-23
8. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
9. ANTHROPIC. *Scaling Managed Agents: Decoupling the brain from the hands* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/managed-agents>
10. YAO, Yilun, Xinyu TAN, Chao-Hsuan LIU, Yaoming LI, Zhengyang WANG, Wenhan YU, Zhewen TAN, Yuxuan TIAN, Guangxiang ZHAO, Lin SUN, Xiangzheng ZHANG a Tong YANG. *Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows*. *arXiv preprint arXiv:2605.27922* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://arxiv.org/abs/2605.27922>. 27. května 2026. 5194 běhových trajektorií

11. CHEN, Tingyang, Shuo LU, Kang ZHAO, Weicheng MENG, Hanlin TENG, Tianhao LI, Chao LI, Xule LIU, Jian LIANG, Zhizhong ZHANG, Yuan XIE, Heng QU, Kun SHAO a Jian LUAN. HarnessX: A Composable, Adaptive, and Evolvable Agent Harness Foundry. *arXiv preprint arXiv:2606.14249* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://arxiv.org/abs/2606.14249>. 12. června 2026, revize 23. července. Průměrně plus 14,5 procent na pěti benchmarcích
12. DING, Fei, Yongkang ZHANG, Runhao LIU, Yuhao LIAO, Zijian ZENG a Huiming YANG. Scaffold-Mediated Post-Training: Co-Evolving Model Parameters and Procedural Scaffold Graphs. *arXiv preprint arXiv:2608.05156* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://arxiv.org/abs/2608.05156>. 22. května 2026. Retence po kompresi scaffoldu 85,2 procent
13. THANGARAJAH, Kishanthan, Boyuan CHEN a Ahmed E. HASSAN. DCAS: Decoupling CLI Agent Scaffolding to Internalize Planning across Scaffolds. *arXiv preprint arXiv:2608.06113* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://arxiv.org/abs/2608.06113>. 6. srpna 2026. Předběžný výtisk; patová hlavička PDF nese ASE 26, znamená na arXiv trefu neuvádí
14. GU, Shangding. From Model Scaling to System Scaling: Scaling the Harness in Agentic AI. *arXiv preprint arXiv:2605.26112* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://arxiv.org/abs/2605.26112>. 25. května 2026. Požiční práce s referenční implementací CheetahClaws, ne měřicí studie
15. MARIUS, Patrik. *DarkFactory* revize *d576ec8f* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://github.com/marius-patrik/DarkFactory/tree/d576ec8fdb78a654bfff9d0da165aa81a97b673>. Poslední revize, v níž ještě pipeline volá produkční harnessy
16. MERRIAM-WEBSTER. *Agentic: Definition & Meaning* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://www.merriam-webster.com/dictionary/agentic>. Původ slova: agent + -ic, poprvé doloženo 1966
17. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need. *arXiv preprint arXiv:1706.03762* [online]. 2017. Dostupné z: <https://arxiv.org/abs/1706.03762>

18. BROWN, Tom B., Benjamin MANN, Nick RYDER, Melanie SUBBIAH, Jared KAPLAN, Prafulla DHARIWAL, Arvind NEELAKANTAN, Pranav SHYAM, Girish SASTRY, Amanda ASKELL a OTHERS. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* [online]. 2020, **33**. Dostupné z: <https://arxiv.org/abs/2005.14165>
19. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>
20. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: <https://arxiv.org/abs/2307.03172>
21. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
22. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: <https://arxiv.org/abs/2210.03629>
23. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: <https://www.anthropic.com/engineering/building-effective-agents>
24. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://agentskills.io/home>
25. OPENAI. *Lifecycle / OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
26. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://modelcontextprotocol.io/specification/2026-07-28/server>

27. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://agents.md/>
28. WILLISON, Simon. *What is agentic engineering?* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://simonwillison.net/guides/agentic-engineering-patterns/what-is-agentic-engineering>. Kapitola průvodce Agentic Engineering Patterns
29. ALENEZI, Mamdouh. Educating the Agentic Engineer: Curricula, Collaboration, and Continuous Learning in the AI Era. *arXiv preprint arXiv:2607.29610* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://arxiv.org/abs/2607.29610>
30. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>
31. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
32. OPENAI. *Using Goals in Codex: Persistent Objectives for Long-Running Work* [online]. 2026 [vid. 2026-09-27]. Dostupné z: https://developers.openai.com/cookbook/examples/codex/using_goals_in_codex
33. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://openai.github.io/openai-agents-python/multi_agent/
34. ANTHROPIC. *Orchestrate subagents at scale with dynamic workflows* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://code.claude.com/docs/en/workflows>. Sekce Dynamic workflows. Převzatá ilustrace rozhraní, nikoli záznam běhu
35. KIMI TEAM. Kimi K2.5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://arxiv.org/abs/2602.02276>
36. HUNTLEY, Geoffrey. *Ralph Wiggum as a „software engineer“* [online]. 2025 [vid. 2026-09-27]. Dostupné z: <https://ghuntley.com/ralph/>. 14. července 2025. Původní text, který techniku pojmenoval
37. ANTHROPIC. *Keep Claude working toward a goal* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://code.claude.com/docs/en/goal>

38. GOOGLE. *Antigravity CLI* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://antigravity.google/product/antigravity-cli>. Sekce Subagents functionalities. Převzatá ilustrace rozhraní, nikoli záznam běhu
39. NATO. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968* [online]. Brussels: NATO, Scientific Affairs Division, 1969 [vid. 2026-09-27]. Dostupné z: <http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF>. 231 s. Zpráva o softwarové krizi; stažitelný přepis z roku 2001 má 136 stran, číslování citací se vztahuje k původnímu vydání
40. COMPUTER HISTORY MUSEUM. *Robert W. Berner* [online]. [vid. 2026-09-27]. Dostupné z: <https://history.computer.org/bemer.html>. Životopis uvádí vynálezce pojmu software factory a jeho působení v General Electric
41. RANDELL, Brian. *Software Engineering in 1968* [online]. 1968 [vid. 2026-09-27]. Dostupné z: <http://homepages.cs.ncl.ac.uk/brian.randell/PapersInProceedings/349.pdf>. University of Newcastle upon Tyne, s. 7, vzpomínka na konferenci v Garmischi
42. BEMER, Robert W. Toward the Complete Software Factory. In: *Honeywell Software Productivity Symposium, Minneapolis, 26–28 April 1977* [online]. 1977 [vid. 2026-09-27]. Dostupné z: <https://archive.computerhistory.org/resources/access/text/2021/04/102785423-05-03-acc.pdf>. Sborník Published Papers, 1951–1999, Computer History Museum, oddíl 6.5.1
43. FACTORY.AI. *Factory 2.0: From coding agents to software factories* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://factory.ai/news/software-factory>. Matan Grinberg a Eno Reyes, 15. června 2026
44. BCG PLATINION. *The Agentic Software Factory* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.bcgplatinion.com/insights/the-agentic-software-factory>
45. GUILD.AI. *Introducing Guild Software Factory* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.guild.ai/blog/product/introducing-guild-software-factory>. 3. září 2026. Číslo jsou vlastní měření společnosti
46. ANTHROPIC. *The 2026 State of AI Agents Report* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://resources.anthropic.com/hubfs/The%202026%20State%20of%20AI%20Agents%20Report.pdf>

47. GITHUB. *Branches / GitHub Docs* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://docs.github.com/en/pull-requests/reference/branches>
48. GITHUB. *About pull requests / GitHub Docs* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
49. MARIUS, Patrik. *DarkFactory* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://github.com/marius-patrik/DarkFactory>. Repozitář projektu, na němž praktická část staví

Seznam součástí textu

Obrázek 1	Doplňování kódu přímo v editoru: model navrhuje pokračování řádku, které člověk přijme nebo odmítne (1).....	5
Obrázek 2	Odhad rozdělení uživatelů generativní AI podle typu (7).....	6
Obrázek 3	Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov <i>král</i> , <i>královna</i> , <i>muž</i> a <i>žena</i> (19).....	10
Obrázek 4	Agentní smyčka ReAct: model navrhne akci, harness ji provede v běhovém prostředí a pozorování se vrací do dalšího kroku. Ukončení nastává, když model místo další akce vydá závěrečnou odpověď (22).....	11
Obrázek 5	Postup práce dynamického workflow v Claude Code: vlevo fáze plánu se stavem, vpravo rozvinutá fáze <i>CodeReview</i> se šestnácti agenty, u každého model, spotřebované tokeny, počet nástrojů a doba běhu. Screenshot pochází z komunitního příspěvku; rozhraní i jeho chování popisuje dokumentace (34).....	14
Obrázek 6	Rozhraní Google Antigravity CLI, tedy harnessu agenta běžícího v terminálu: koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem (38).....	15
Obrázek 7	Architektura: GitHub poskytuje události a vývojový stav, GitHub Actions výpočet, Docker odděluje běh a pythonovský runner převádí událost na agentní krok. Produkční harnessy zajišťují model, nástroje a pozorování; registr jich má osm, obrázek ukazuje čtyři (15).....	22
Obrázek 8	Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka se opakuje, dokud nález nezmizí, v této revizi nejvýše třikrát (15).....	22
Tabulka 1	Předloha požadavku a nejstarší požadavek projektu, issue 727 z 20. září 2026. Předloha má šest polí, z nichž tři povinná. Skutečný požadavek však předlohu nevyplnil, ale napsal volně, pod nadpisy <code>Problem</code> a <code>Contract</code> ; tabulka proto ukazuje tři povinná pole předlohy a to, jak požadavek ve skutečnosti vypadá. Runner předává modelu titulky i tělo issue, takže právě to rozhoduje, co se do zadání dostane (15).....	25